

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 05 -

ESCALABILIDADE DE BANCO DE DADOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	9
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMANIP

O QUE VEM POR AÍ?

Até agora, escalamos com sucesso nossa aplicação, distribuímos o tráfego com LoadBalancers e aceleramos a entrega de conteúdo globalmente com CDNs. Nossos sistemas são resilientes e seguem os melhores padrões. No entanto, quase todas as ações significativas que um usuário realiza, uma compra, um post, uma busca, precisam, em algum momento, ler ou escrever dados. E todos os nossos componentes escaláveis geralmente apontam para um único lugar, o guardião da verdade do nosso sistema: o banco de dados. Este componente é, para muitas arquiteturas, o “chefão” final da escalabilidade.

Nesta aula, vamos enfrentar de frente este que é um dos maiores desafios da engenharia de software. Você entenderá por que é tão difícil escalar um banco de dados, dada a sua natureza inerentemente "stateful" e a necessidade crítica de manter a consistência dos dados. Vamos desvendar as estratégias para escalar desde o primeiro gargalo comum, a carga de leitura, até o problema mais complexo de todos: a “carga de escrita”. Apresentaremos os teoremas fundamentais que governam os sistemas distribuídos e as métricas de negócio que definem o quanto resiliente sua estratégia de dados realmente precisa ser.

HANDS ON

Nesta seção, faremos uma exploração conceitual das funcionalidades de escalabilidade e resiliência oferecidas pelos serviços de banco de dados em nuvem. O objetivo é que você veja como os padrões que discutiremos são implementados na prática, por meio das interfaces e configurações que os provedores de nuvem nos oferecem. As videoaulas que acompanham este material guiarão você por essa demonstração.

Começaremos no painel do Amazon RDS (RelationalDatabase Service). Demonstraremos como, a partir de uma instância de banco de dados principal já existente, é possível criar uma Read Replica (Réplica de Leitura) com apenas alguns cliques. Vamos destacar como a AWS gerencia a replicação dos dados de forma assíncrona e como a nova instância de réplica aparece pronta para receber tráfego de leitura, ilustrando a simplicidade de implementar este poderoso padrão de escalabilidade.

Em seguida, vamos trabalhar com pseudocódigo para demonstrar o padrão de caching "cache-aside". Escreveremos uma função simples que, antes de consultar o banco de dados, primeiro tenta buscar a informação de um serviço de cache como o Redis. Se a informação não estiver lá (um "cache miss"), a função busca no banco de dados principal e, crucialmente, armazena o resultado no cache antes de retorná-lo. Este exercício prático mostrará como o caching alivia a carga do banco de dados. Por fim, faremos uma breve incursão em um banco de dados NoSQL como o AmazonDynamoDB, mostrando como sua natureza horizontalmente escalável é configurada por meio da definição de "unidades de capacidade de leitura e escrita".

SAIBA MAIS

O banco de dados é o coração de quase toda aplicação, é o repositório central. Diferente dos servidores de aplicação, que idealmente são stateless, o banco de dados é, por definição, stateful. Essa natureza, combinada com a necessidade de garantir a integridade dos dados, torna sua escalabilidade um desafio único e complexo. Os principais desafios que enfrentamos são a consistência (garantir que uma transação seja aplicada de forma atômica e que todas as leituras subsequentes vejam o resultado correto), a concorrência (gerenciar múltiplos acessos simultâneos sem corromper os dados) e a latência (manter as operações rápidas mesmo com o aumento do volume de dados e requisições).

Geralmente, o primeiro gargalo a surgir em um banco de dados é a carga de leitura. Em muitas aplicações, a proporção de leituras para escritas é muito alta (ex: 100 leituras para cada 1 escrita). A primeira linha de defesa contra esse gargalo é o Caching. Ao colocar uma camada de cache em memória de alta velocidade, como Redis ou Memcached, entre a aplicação e o banco de dados, podemos servir as requisições de leitura mais frequentes sem tocar no banco, reduzindo drasticamente a latência e a carga.

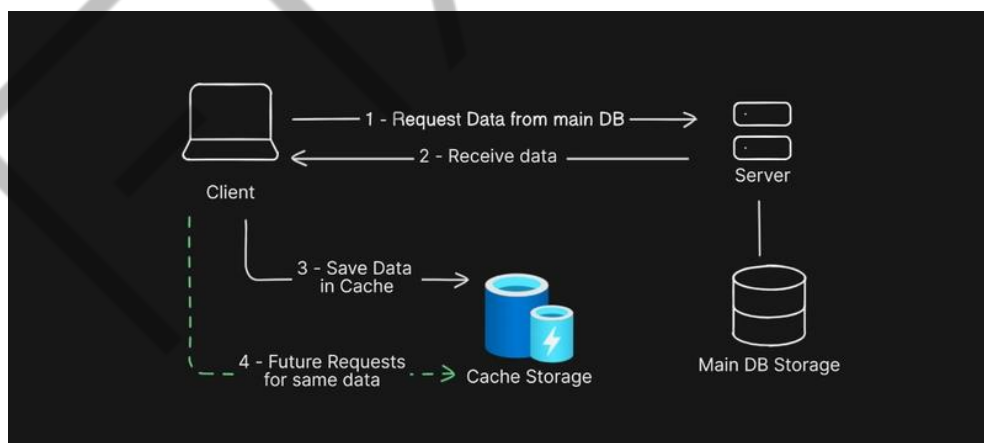


Figura 1 - Cache em banco de dados

Fonte: <https://dev.to/paras594/basics-of-caching-139l> (2024)

Quando o cache não é suficiente, a próxima estratégia é a criação de Read Replicas (Réplicas de Leitura). Este padrão utiliza um modelo de replicação "master-slave" (ou "primary-replica"). Temos um banco de dados principal (Primary) que aceita todas as operações de escrita (INSERT, UPDATE, DELETE). Os dados do

Primary são então replicados, geralmente de forma assíncrona, para uma ou mais cópias somente leitura (Réplicas). A aplicação é então modificada para implementar o Write Splitting: todas as escritas são direcionadas ao Primary, enquanto as leituras são distribuídas entre as várias réplicas. Isso permite escalar a capacidade de leitura de forma horizontal, simplesmente adicionando mais réplicas.

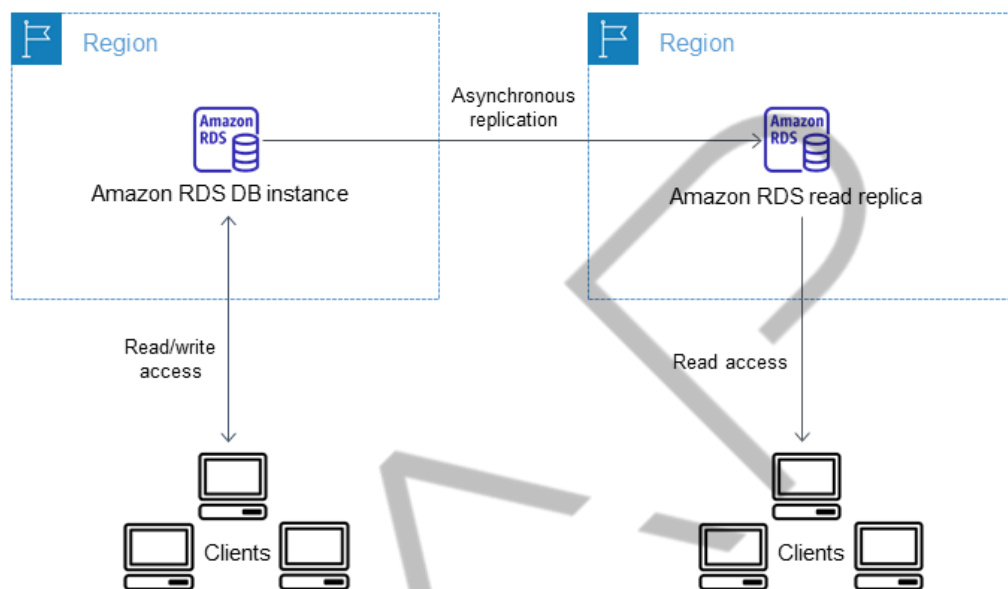


Figura 2 - Réplica de Leitura
Fonte: [Amazon](#) (s.d)

Contudo, essa abordagem tem um limite: todas as escritas ainda são direcionadas a um único servidor. Quando a carga de escrita se torna o gargalo, o problema se torna muito mais complexo de resolver. A principal estratégia para escalar a escrita é o Sharding (também conhecido como particionamento horizontal). O Sharding divide um banco de dados grande em múltiplos bancos menores e independentes, os "shards". A lógica de como dividir os dados é crucial. Uma estratégia de sharding por range (ex: usuários com nomes de A-M no Shard 1, N-Z no Shard 2) é simples, mas pode criar "hotspots" se a distribuição não for uniforme. Uma estratégia de sharding por hash (aplicando uma função de hash a uma chave, como o ID do usuário) distribui os dados de forma mais equitativa, mas pode dificultar consultas que precisam varrer uma faixa de dados. O Sharding é extremamente poderoso, mas adiciona uma complexidade imensa à aplicação e à operação, especialmente quando é necessário um "re-sharding".

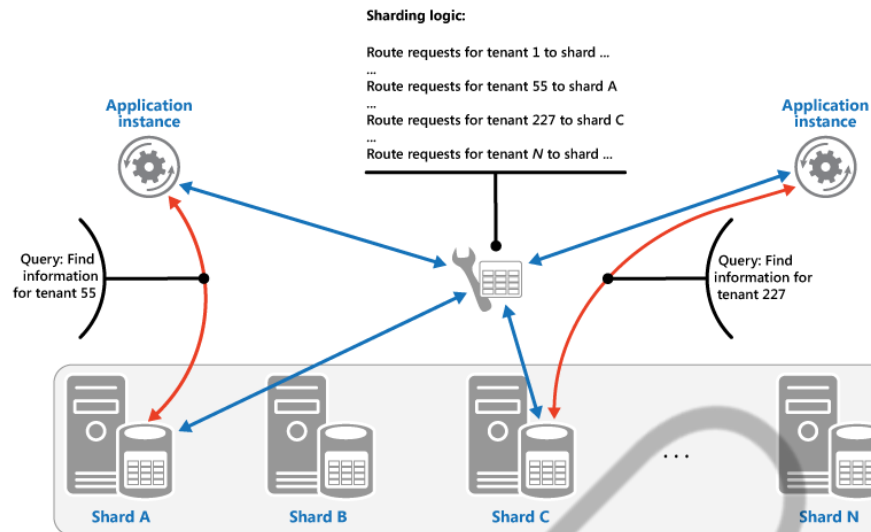


Figura 3 - Sharding em banco de dados

Fonte: [Microsoft](#) (s.d)

Para evitar essa complexidade manual, surgiu uma nova geração de Bancos de Dados Distribuídos, projetados desde o início para escalar horizontalmente. Sistemas como AmazonDynamoDB (NoSQL), CockroachDB (NewSQL) e Apache Cassandra (NoSQL) gerenciam o sharding e a replicação de forma transparente para a aplicação. Eles foram construídos sobre os princípios de sistemas distribuídos, o que nos leva à teoria fundamental que os governa.

O Teorema CAP, formulado por Eric Brewer, afirma que um sistema distribuído só pode garantir, simultaneamente, duas de três propriedades: **Consistência** (todos os nós veem os mesmos dados ao mesmo tempo), **A disponibilidade** (todas as requisições recebem uma resposta, sem garantia de que contenha a escrita mais recente) e **Partição de Tolerância** (o sistema continua a operar mesmo que haja uma quebra na comunicação entre os nós).

Como falhas de rede (partições) são uma realidade inevitável, a escolha real é sempre entre Consistência e Disponibilidade (CP ou AP). Bancos de dados relacionais tradicionais geralmente escolhem CP. Muitos bancos NoSQL, como o Cassandra, escolhem AP, o que nos leva ao modelo de Consistência Eventual. Nesse modelo, o sistema garante que, se nenhuma nova atualização for feita a um dado, eventualmente todas as réplicas convergirão para o mesmo valor. Isso significa que, por um breve período, leituras em diferentes nós podemos retornar valores diferentes, uma característica que os desenvolvedores precisam levar em conta ao projetar a aplicação.

Com tantas opções, a estratégia de escalabilidade de um banco de dados deve ser bem pensada. A decisão de escalar verticalmente vs. horizontalmente ainda é relevante. Escalar verticalmente (comprar uma máquina maior) pode ser a solução mais simples e eficaz para muitas cargas de trabalho até um certo ponto, evitando a complexidade da distribuição. A escala horizontal é mais complexa, mas oferece um teto de escala muito maior e maior resiliência.

A resiliência é, talvez, o aspecto mais crítico. Uma estratégia de failover define o que acontece quando o banco de dados principal falha. Em uma configuração com réplicas, um processo de failover automatizado pode "promover" uma das réplicas a se tornar o novo nó primário, minimizando o tempo de inatividade. A robustez dessa estratégia é medida por duas métricas de negócio:

RPO (Recovery Point Objective): o ponto no tempo para o qual precisamos conseguir recuperar, ou seja, quanta perda de dados o negócio pode tolerar? Um RPO de 5 minutos significa que, em caso de desastre, a empresa aceita perder os últimos 5 minutos de transações.

RTO (Recovery Time Objective): o tempo total que o sistema pode permanecer indisponível após um desastre, ou seja, por quanto tempo podemos ficar fora do ar? Um RTO de 15 minutos significa que todo o processo de detecção da falha, failover e retorno à operação deve ser concluído em menos de 15 minutos.

Essas duas métricas, RPO e RTO, são definidas pelo negócio e ditam as escolhas técnicas de replicação (síncrona vs. assíncrona) e automação de failover.

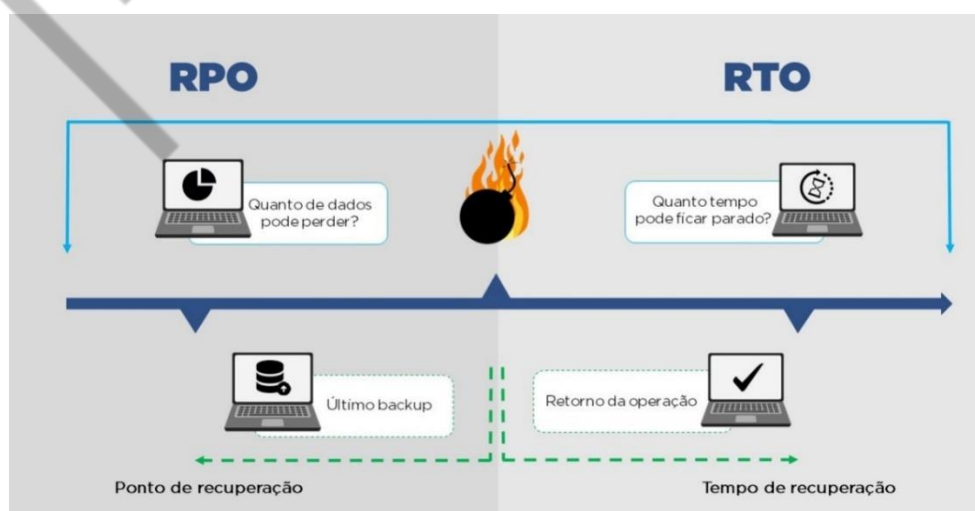


Figura 4 - RPO vs RTO
Fonte: [CMM](#) (2020)

MERCADO, CASES E TENDÊNCIAS

A inovação no mundo dos bancos de dados é constante, com novas arquiteturas surgindo para resolver os desafios de escala e consistência em um mundo cada vez mais distribuído.

Case: Google Spanner e os Bancos de Dados "NewSQL"

Por muito tempo, o Teorema CAP ditou uma escolha difícil entre consistência e disponibilidade. O Google Spanner foi um divisor de águas ao provar que era possível construir um banco de dados globalmente distribuído que oferecia consistência forte (similar a um banco relacional) e alta disponibilidade. Ele faz isso utilizando hardware especializado, como relógios atômicos, para sincronizar transações ao redor do mundo. Empresas como a Cockroach Labs (com o CockroachDB) estão trazendo essa mesma capacidade "NewSQL" para o mercado mais amplo, permitindo que empresas construam aplicações globalmente consistentes sem o hardware do Google.

Tendência: A Ascensão dos Bancos de Dados Serverless

A próxima fronteira da escalabilidade de bancos de dados é a abstração total da infraestrutura. Serviços como o Amazon Aurora Serverless, MongoDB Atlas Serverless e Neon (para Postgres) estão ganhando tração. Eles separam completamente a computação do armazenamento e escalam a capacidade de processamento de forma automática e instantânea, baseada na carga de trabalho da aplicação. Para aplicações com tráfego intermitente ou imprevisível, isso é revolucionário, pois a capacidade pode escalar até zero, e o custo acompanha o uso real, eliminando a necessidade de provisionar e pagar por servidores ociosos.

Notícia: A Convergência com HTAP (Hybrid Transactional/Analytical Processing)

Tradicionalmente, as empresas mantinham bancos de dados separados para suas operações do dia a dia (OLTP) e para análises de dados (OLAP). Isso exigia processos complexos de ETL (Extract, Transform, Load) para mover os dados. Uma forte tendência é o surgimento de bancos de dados HTAP, como o SingleStore e o TiDB, que são projetados para lidar com ambas as cargas de trabalho — transações de alta frequência e queries analíticas complexas — em um único sistema. Isso

simplifica a arquitetura de dados e permite análises em tempo real sobre os dados mais recentes da operação.

EMEND

O QUE VOCÊ VIU NESTA AULA?

Nesta aula densa e fundamental, enfrentamos o desafio da escalabilidade de bancos de dados. Você aprendeu sobre os desafios centrais de consistência, concorrência e latência. Vimos as estratégias para o primeiro gargalo e a carga de leitura por meio de Caching e Read Réplicas.

Em seguida, abordamos o problema mais complexo da escala de escrita com o Sharding. Mergulhamos na teoria que governa os sistemas modernos com o Teorema CAP e o conceito de Consistência Eventual. Por fim, trouxemos a discussão para o mundo dos negócios, entendendo a importância das estratégias de failover e como as métricas RPO e RTO ditam a arquitetura de resiliência de dados da sua aplicação.

REFERÊNCIAS

AWS. **Amazon Aurora Features: High Availability and Durability**. Amazon Web Services Documentation. Disponível em: <https://aws.amazon.com/rds/aurora/features/>. Acesso em: 28 ago. 2025.

COCKROACH LABS. **CAP Theorem**. CockroachDB Docs. Disponível em: <https://www.cockroachlabs.com/docs/stable/architecture/overview#cockroachdb-architecture-terms>. Acesso em: 28 ago. 2025.

KLEPPMANN, M. **Designing Data-Intensive Applications**. Sebastopol: O'Reilly Media, 2017.

PALAVRAS-CHAVE

Consistencia. Caching. Read Replica. Failover. RPO. RTO.

EMEND



POSTECH